

LAPORAN PRAKTIKUM
THREADS & CONCURRENCY



Dosen Pengampu : Triawan Adi Cahyanto, M.K

Di susun Oleh:

Muhammad maulana wiramatajaya 2410651006

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2024

1. EKSPERIMEN CPU SHEDULING

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

- P1: AT=0, BT=10
- P2: AT=1, BT=5
- P3: AT=2, BT=8

Data Set 2:

- P1: AT=0, BT=5
- P2: AT=1, BT=3
- P3: AT=2, BT=8
- P4: AT=3, BT=6

Hasil Percobaan dengan menggunakan data set 1:

• First Come First Served	• Shortest Job First
<pre>wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_4\$./fcfs === SIMULASI ALGORITMA FCFS === Masukkan jumlah proses : 3 Process ID : 1 Arrival Time: 0 Burst Time: 10 Process ID : 2 Arrival Time: 1 Burst Time: 5 Process ID : 3 Arrival Time: 2 Burst Time: 8 === HASIL PENJADWALAN FCFS === PID AT BT CT TAT WT --- -- -- -- -- -- P1 0 10 10 10 0 P2 1 5 15 14 9 P3 2 8 23 21 13 Average Turnaround Time: 15.00 Average Waiting Time: 7.33 === GANTT CHART === P1 P2 P3 0 10 15 23</pre>	<pre>=== SIMULASI ALGORITMA SJF (non-preemptive) === Masukkan jumlah proses: 3 Proses P1 Arrival Time: 0 Burst Time: 10 Proses P2 Arrival Time: 1 Burst Time: 5 Proses P3 Arrival Time: 2 Burst Time: 8 === HASIL PENJADWALAN SJF === PID AT BT CT TAT WT --- -- -- -- -- -- P1 0 10 10 10 0 P2 1 5 15 14 9 P3 2 8 23 21 13 Average Turnaround Time: 15.00 Average Waiting Time: 7.33 === URUTAN EKSEKUSI === Proses dieksekusi dalam urutan: P1 -> P2 -> P3 wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_4\$</pre>

Hasil percobaan dengan menggunakan data set 2 :

• First Come First Served	• Shortest Job First
<pre>=== SIMULASI ALGORITMA FCFS === Masukkan jumlah proses : 4 Process ID : 1 Arrival Time: 0 Burst Time: 5 Process ID : 2 Arrival Time: 1 Burst Time: 3 Process ID : 3 Arrival Time: 2 Burst Time: 8 Process ID : 4 Arrival Time: 3 Burst Time: 6 === HASIL PENJADWALAN FCFS === PID AT BT CT TAT WT --- -- -- -- -- -- P1 0 5 5 5 0 P2 1 3 8 7 4 P3 2 8 16 14 6 P4 3 6 22 19 13 Average Turnaround Time: 11.25 Average Waiting Time: 5.75 === GANTT CHART === P1 P2 P3 P4 0 5 8 16 22</pre>	<pre>=== SIMULASI ALGORITMA SJF (non-preemptive) === Masukkan jumlah proses: 3 Proses P1 Arrival Time: 0 Burst Time: 10 Proses P2 Arrival Time: 1 Burst Time: 5 Proses P3 Arrival Time: 2 Burst Time: 8 === HASIL PENJADWALAN SJF === PID AT BT CT TAT WT --- -- -- -- -- -- P1 0 10 10 10 0 P2 1 5 15 14 9 P3 2 8 23 21 13 Average Turnaround Time: 15.00 Average Waiting Time: 7.33 === URUTAN EKSEKUSI === Proses dieksekusi dalam urutan: P1 -> P2 -> P3 wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_4\$</pre>

2. Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban :

1. Algoritma mana yang memberikan Average Waiting Time (Waktu Tunggu Rata-rata) terkecil?

Algoritma **Shortest Job First (SJF)** secara umum memberikan **Average Waiting Time (AWT)** terkecil untuk sekumpulan proses tertentu.

- SJF bekerja dengan memprioritaskan proses yang memiliki waktu eksekusi (*burst time*) terpendek, sehingga proses-proses pendek cepat selesai dan keluar dari sistem, yang secara signifikan mengurangi total waktu tunggu dan waktu tunggu rata-rata proses lainnya.
- Berdasarkan studi perbandingan, SJF (terutama yang *preemptive* atau Shortest Remaining Time First/SRTF) sering kali memiliki AWT yang lebih rendah dibandingkan FCFS dan Round Robin (RR).

2. Mengapa hasil algoritma berbeda-beda?

Hasil ketiga algoritma berbeda-beda karena **perbedaan mendasar dalam kriteria dan strategi penjadwalan** yang digunakan:

- **FCFS (First-Come, First-Served):** Kriterianya adalah **waktu kedatangan** (*arrival time*). Proses dilayani berdasarkan urutan kedatangan, mirip antrian biasa. Ini bersifat *non-preemptive* (tidak dapat disela).
- **SJF (Shortest Job First):** Kriterianya adalah **waktu eksekusi terpendek** (*shortest burst time*). Algoritma ini bertujuan untuk optimalisasi waktu tunggu. Dapat bersifat *non-preemptive* atau *preemptive* (SRTF).
- **Round Robin (RR):** Kriterianya adalah **pembagian waktu** (*time quantum*). Algoritma ini bersifat *preemptive* dan memberikan jatah waktu eksekusi yang sama secara bergiliran kepada setiap proses, menciptakan rasa keadilan.

Strategi yang berbeda ini menghasilkan kinerja yang berbeda pula pada metrik seperti Average Waiting Time, *Turnaround Time*, dan *Response Time*.

3. Kapan sebaiknya menggunakan algoritma FCFS?

Algoritma FCFS sebaiknya digunakan dalam situasi berikut:

- **Sistem *Batch* Non-Interaktif:** Cocok untuk sistem di mana pekerjaan diproses secara sekuensial dan **keadilan sederhana** (siapa datang duluan dilayani duluan) menjadi prioritas, bukan kecepatan respons.
- **Kasus dengan *Burst Time* yang Seragam:** FCFS dapat memberikan hasil yang cukup baik jika semua proses memiliki waktu eksekusi yang kurang lebih sama.
- **Kesederhanaan Implementasi:** Digunakan ketika dibutuhkan algoritma yang **paling mudah dipahami dan diimplementasikan** dengan *overhead* yang minimal.

4. Kapan sebaiknya menggunakan algoritma SJF?

Algoritma SJF sebaiknya digunakan dalam situasi berikut:

- **Tujuan Meminimalkan Waktu Tunggu Rata-rata:** SJF adalah algoritma optimal untuk mendapatkan **AWT** dan **Turnaround Time** rata-rata terkecil.
- **Sistem Batch (SJF Non-Preemptive):** Ketika kita dapat mengetahui perkiraan waktu eksekusi proses dan ingin memproses proses-proses kecil secepatnya.
- **Sistem Time-Sharing atau Real-Time (SJF Preemptive/SRTF):** Ketika proses baru yang sangat singkat tiba dan harus segera dieksekusi untuk meminimalkan waktu tunggu proses-proses lain. **Kelemahan utamanya adalah sulitnya memprediksi burst time proses secara akurat** dan adanya risiko **kelaparan (starvation)** untuk proses yang sangat panjang.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Algoritma Round Robin sebaiknya digunakan dalam situasi berikut:

- **Sistem Interaktif atau Time-Sharing:** Ini adalah pilihan ideal untuk sistem operasi modern yang bersifat **multitasking** (seperti komputer pribadi atau server multi-pengguna) di mana **responsivitas yang cepat** dan **pembagian waktu yang adil** antar pengguna atau proses adalah kuncinya.
- **Menghindari Starvation:** Karena setiap proses mendapatkan jatah waktu secara bergiliran, RR menjamin bahwa semua proses pada akhirnya akan dieksekusi, sehingga **mencegah kelaparan (starvation)**.
- **Waktu Respons Cepat:** RR unggul dalam memberikan *Response Time* yang lebih baik daripada FCFS dan SJF non-preemptive, karena semua proses mulai mendapatkan eksekusi relatif cepat.

3. Modifikasi Program

A. Tugas

Tambahkan fitur berikut pada program FCFS:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

B. Langkah Langkah

Tambahkan 3 method pada program fcfs yang kita sudah buat

```

// Fungsi untuk membaca input dari file
void read_input_from_file(struct Process proc[], int *n) {
    FILE *file;
    char filename[] = "input_fcfs.txt";

    file = fopen(filename, "r");
    if (file == NULL) {
        printf("ERROR: File '%s' tidak ditemukan atau gagal dibuka.\n", filename);
        printf("Pastikan file tersebut ada dan memiliki format:\n");
        printf("Jumlah_Proses\n");
        printf("AT_P1 BT_P1\n");
        printf("AT_P2 BT_P2\n");
        printf("...\n");
        exit(1);
    }

    // Baca jumlah proses
    if (fscanf(file, "%d", n) != 1 || *n <= 0) {
        printf("ERROR: Format file input salah pada baris jumlah proses.\n");
        fclose(file);
        exit(1);
    }

    printf("Membaca %d proses dari file '%s'...\n", *n, filename);

    // Baca data proses
    for (int i = 0; i < *n; i++) {
        proc[i].pid = i + 1;
        // Baca Arrival Time dan Burst Time
        if (fscanf(file, "%d %d", &proc[i].arrival_time, &proc[i].burst_time) != 2) {
            printf("ERROR: Format data proses ke-%d salah.\n", i + 1);
            fclose(file);
            exit(1);
        }
    }

    fclose(file);
}

```

```

73 // Fungsi untuk menampilkan Gantt Chart yang lebih detail
74 void display_gantt_chart(struct Process proc[], int n) {
75     printf("\n=== GANTT CHART DETAIL ===\n");
76     // Baris 1: Batas atas dan nama proses
77     printf(" ");
78     for (int i = 0; i < n; i++) {
79         // Lebar cell adalah burst time
80         int width = proc[i].burst_time;
81         for (int j = 0; j < width; j++) printf("--");
82         printf(" ");
83     }
84     printf("\n|");
85     // Tampilkan ID Proses
86     for (int i = 0; i < n; i++) {
87         int width = proc[i].burst_time;
88         // Hitung padding untuk menengahkan Px
89         int pad_left = (width * 2 - 2) / 2;
90         int pad_right = width * 2 - 2 - pad_left;
91
92         // Cek dan cetak waktu idle jika ada
93         if (i == 0 && proc[i].arrival_time > 0) {
94             printf(" Idle ");
95         } else if (i > 0 && proc[i].arrival_time > proc[i-1].completion_time) {
96             // Waktu idle antara proses
97             printf(" Idle ");
98         }
99         printf("%sP%d%s|", pad_left, "", proc[i].pid, pad_right, "");
100     }
101     printf("\n ");
102
103     // Baris 2: Batas bawah dan waktu
104     for (int i = 0; i < n; i++) {
105         int width = proc[i].burst_time;
106         for (int j = 0; j < width; j++) printf("--");
107         printf(" ");
108     }
109     printf("\n");
110     // Baris 3: Waktu selesai
111     int time_cursor = 0;
112     printf("%d", time_cursor);
113
114     for (int i = 0; i < n; i++) {
115         // Tambahkan spasi untuk idle time jika ada
116         if (i == 0 && proc[i].arrival_time > 0) {
117             time_cursor = proc[i].arrival_time;
118             // Tambahkan spasi untuk waktu idle
119             printf("%s%d", (proc[i].arrival_time) * 2 - 1, "", time_cursor);
120         } else if (i > 0 && proc[i].arrival_time > proc[i-1].completion_time) {
121             int idle_time = proc[i].arrival_time - proc[i-1].completion_time;
122             time_cursor = proc[i].arrival_time;
123             // Tambahkan spasi untuk waktu idle
124             printf("%s%d", (idle_time) * 2 + 1, "", time_cursor);
125         }
126         // Tambahkan spasi untuk burst time
127         int width = proc[i].burst_time;
128         time_cursor = proc[i].completion_time;
129         printf("%s%d", width * 2 + 1, "", time_cursor);
130     }
131     printf("\n");
132 }

```

Lalu compile seperti biasa, setelah di compile masukan buat file fcfs.txt

Dan masukan text seperti format di bawah ini

```
input_fcfs.txt
1 4
2 0 5
3 2 3
4 4 8
5 6 4
```

Jika di run maka hasilnya akan seperti di bawah ini

```
=== HASIL PERHITUNGAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     5     5     5     0
P2      2     3     8     6     3
P3      4     8    16    12     4
P4      6     4    20    14    10

Average Turnaround Time: 9.25
Average Waiting Time: 4.25

=== SIMULASI ALGORITMA FCFS ===
Membaca 4 proses dari file 'input_fcfs.txt'...

=== HASIL PERHITUNGAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     5     5     5     0
P2      2     3     8     6     3
P3      4     8    16    12     4
P4      6     4    20    14    10

Average Turnaround Time: 9.25
Average Waiting Time: 4.25

P1      0     5     5     5     0
P2      2     3     8     6     3
P3      4     8    16    12     4
P4      6     4    20    14    10

Average Turnaround Time: 9.25
Average Waiting Time: 4.25

Average Turnaround Time: 9.25
Average Waiting Time: 4.25

Average Turnaround Time: 9.25
Average Waiting Time: 4.25

=== GANTT CHART DETAIL ===
-----
|  P1  |  P2  |    P3    |  P4  |
-----
0      5      8      16      20
```

4. Eksperimen dengan data berbeda untuk perbandingan algoritma

1. Proses dengan burst time bervariasi

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_4$ ./Round_Robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3

Proses P1
Arrival Time: 0
Burst Time: 24

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 3

Proses P4
Arrival Time: 3
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 21
Waktu 3: Menjalankan P1 -> Sisa waktu: 18
Waktu 6: Menjalankan P2 -> Selesai pada waktu 9
Waktu 9: Menjalankan P3 -> Selesai pada waktu 12
Waktu 12: Menjalankan P4 -> Sisa waktu: 3
Waktu 15: Menjalankan P1 -> Sisa waktu: 15
Waktu 18: Menjalankan P1 -> Sisa waktu: 12
Waktu 21: Menjalankan P1 -> Sisa waktu: 9
Waktu 24: Menjalankan P4 -> Selesai pada waktu 27
Waktu 27: Menjalankan P4 -> Selesai pada waktu 27

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT         | TAT | WT |
|-----|----|----|------------|-----|----|
| --- | -- | -- | --         | --- | -- |
| P1  | 0  | 24 | 1775623827 | 0   | 0  |
| P2  | 1  | 3  | 9          | 8   | 5  |
| P3  | 2  | 3  | 12         | 10  | 7  |
| P4  | 3  | 6  | 27         | 24  | 18 |


Average Turnaround Time: 10.50
Average Waiting Time: 7.50
```


2. Proses datang bersamaan

```
wiramatajaya@LAPTOP-7838G7J9:~/pertemuan_4$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 0
Burst Time: 5

Proses P3
Arrival Time: 0
Burst Time: 8

Proses P4
Arrival Time: 0
Burst Time: 12

Proses P5
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 4: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P4 -> Sisa waktu: 8
Waktu 16: Menjalankan P5 -> Sisa waktu: 2
Waktu 20: Menjalankan P1 -> Sisa waktu: 2
Waktu 24: Menjalankan P1 -> Selesai pada waktu 26
Waktu 26: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P3 -> Selesai pada waktu 31
Waktu 31: Menjalankan P3 -> Selesai pada waktu 31

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT    BT    CT    TAT    WT
---   --    --    --    ---    --
P1     0     10    26     26     16
P2     0      5    27     27     22
P3     0      8    31     31     23
P4     0     12   32765  -1533811632  32765
P5     0      6  -2090331558    0      0

Average Turnaround Time: -306762304.00
Average Waiting Time: 6565.20
```

3. Proses datang dengan interval

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_4$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 2

Proses P1
Arrival Time: 0
Burst Time: 8

Proses P2
Arrival Time: 2
Burst Time: 4

Proses P3
Arrival Time: 4
Burst Time: 9

Proses P4
Arrival Time: 6
Burst Time: 5

Proses P5
Arrival Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 2: Menjalankan P1 -> Sisa waktu: 4
Waktu 4: Menjalankan P2 -> Sisa waktu: 2
Waktu 6: Menjalankan P1 -> Sisa waktu: 2
Waktu 8: Menjalankan P1 -> Selesai pada waktu 10
Waktu 10: Menjalankan P3 -> Sisa waktu: 7
Waktu 12: Menjalankan P1 -> Selesai pada waktu 12
Waktu 12: Menjalankan P2 -> Selesai pada waktu 14
Waktu 14: Menjalankan P4 -> Sisa waktu: 3
Waktu 16: Menjalankan P2 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT         | TAT         | WT          |
|-----|----|----|------------|-------------|-------------|
| P1  | 0  | 8  | 16         | 16          | 8           |
| P2  | 2  | 4  | 16         | 14          | 10          |
| P3  | 4  | 9  | -276525048 | 28936       | -1103890784 |
| P4  | 6  | 5  | 32766      | -1543178864 | 32766       |
| P5  | 8  | 3  | -276528550 | 0           | 0           |


Average Turnaround Time: -308629984.00
Average Waiting Time: -220771616.00
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_4$
```

5. Analisis Eksperimen perbandingan algoritma

1. Analisis Performa

FCFS (First-Come, First-Served) memberikan hasil terbaik (Average Waiting Time / AWT terkecil, mendekati algoritma lain) ketika:

- **Semua proses memiliki *Burst Time* (BT) yang hampir sama.** Ketika semua proses memiliki BT yang seragam, urutan eksekusi tidak terlalu memengaruhi waktu tunggu secara keseluruhan.
- **Proses dengan BT kecil datang lebih dulu.** Jika proses-proses pendek datang pertama, FCFS akan menyelesaikannya dengan cepat, menghasilkan AWT yang rendah.
- **Tidak ada *Convoy Effect*.** FCFS paling menderita oleh *Convoy Effect*, di mana proses dengan BT yang sangat besar (seperti P1: BT=24 di Test Case 1) datang di awal, memaksa semua proses kecil menunggu lama. FCFS akan memberikan hasil terbaik ketika *Convoy Effect* ini tidak terjadi.

Mengapa SJF hampir selalu memberikan Average WT terkecil?

SJF (Shortest Job First) hampir selalu memberikan **Average Waiting Time (AWT) terkecil** karena SJF adalah algoritma **optimal** dalam hal ini.

- Strategi dasarnya adalah **menyelesaikan proses yang membutuhkan waktu CPU paling sedikit terlebih dahulu.**
- Dengan segera mengeluarkan proses-proses pendek dari antrian, SJF **meminimalkan waktu tunggu** yang harus ditanggung oleh sejumlah besar proses, sehingga AWT keseluruhan menjadi yang terendah.

Apa *trade-off* dari Round Robin?

Trade-off utama dari Round Robin (RR) adalah antara **Waktu Respons (Response Time)** yang cepat dan **Waktu *Overhead* Context Switching** yang tinggi.

Keuntungan (Good)	Kerugian (Bad)
Good Fairness & Response Time: RR menjamin setiap proses mendapat CPU secara teratur, memberikan responsivitas cepat pada sistem interaktif.	High Overhead: RR menghasilkan jumlah <i>context switch</i> yang paling banyak di antara ketiga algoritma, yang menyia-nyiaikan waktu CPU dan meningkatkan <i>overhead</i> sistem.
No Starvation: Setiap proses akan dieksekusi, tidak peduli seberapa panjang <i>Burst Time</i> -nya.	Performance Sensitivity to Quantum: Kinerja sangat bergantung pada ukuran <i>Time Quantum</i> yang tepat. Jika salah, hasilnya bisa buruk (AWT tinggi).

2. Context Switching

Algoritma mana yang paling banyak melakukan *context switch*?

Algoritma **Round Robin (RR)** adalah yang paling banyak melakukan *context switch*.

- Setiap kali jatah waktu (*time quantum*) habis atau proses selesai, sistem harus melakukan *context switch* untuk beralih ke proses berikutnya di antrian *Ready*.
- Semakin kecil *time quantum*, semakin sering *context switch* terjadi.

Bagaimana pengaruh *quantum* terhadap jumlah *context switch* di Round Robin?

- **Quantum Kecil:** $\Downarrow$ **Jumlah *Context Switch* $\Uparrow$ (Tinggi).** Jika *quantum* sangat kecil (misalnya 1), setiap proses akan berganti *state* setelah setiap unit waktu, menghasilkan *overhead* yang sangat besar.

- **Quantum Besar:** $\downarrow$ **Jumlah Context Switch** $\downarrow$ **(Rendah)**. Jika *quantum* sangat besar, RR akan berperilaku mirip dengan FCFS, karena proses yang baru masuk mungkin akan selesai sebelum *quantum*-nya habis.

Coba jalankan Round Robin dengan *quantum* 2, 4, 8, 16 - apa yang terjadi?

Secara umum:

- **Quantum Kecil (2, 4):** Memberikan **respon yang sangat cepat (Response Time)** karena proses segera mendapatkan CPU. Namun, **AWT dan overhead context switch akan tinggi**. Untuk Test Case 3 (*Quantum*=2), *context switch* sangat intens, namun memastikan semua proses (terutama P2, P4, P5 yang relatif pendek) mendapat jatah eksekusi cepat.
- **Quantum Sedang (8):** Menghasilkan **trade-off yang seimbang**. *Overhead context switch* berkurang, sementara *Response Time* masih cukup baik. Ini sering menjadi pilihan optimal.
- **Quantum Besar (16):** Perilaku RR akan **mendekati FCFS**. Jika *quantum* lebih besar dari sebagian besar *Burst Time*, proses-proses tersebut akan selesai dalam satu *slice* waktu, sehingga *context switch* hanya terjadi ketika proses selesai, mirip FCFS.

3. Fairness

Algoritma mana yang paling "adil" untuk semua proses?

Algoritma **Round Robin (RR)** adalah yang paling "adil" (Fair).

- Keadilan di sini didefinisikan sebagai **perlakuan yang merata** atau **pembagian waktu CPU yang sama** untuk semua proses, tanpa memandang *Arrival Time* atau *Burst Time*.
- RR menjamin bahwa tidak ada proses yang akan "kelaparan" dan setiap proses akan mendapat kesempatan eksekusi dalam waktu yang singkat.

Proses mana yang paling dirugikan di algoritma FCFS?

Proses yang paling dirugikan di algoritma FCFS adalah **proses dengan *Burst Time* (BT) kecil yang datang setelah proses dengan BT yang sangat besar**.

- Contohnya di **Test Case 1**, P2, P3, dan P4 dirugikan karena harus menunggu P1 dengan BT=24 selesai terlebih dahulu (meskipun P1 datang paling awal). Proses P2 (BT=3) harus menunggu hingga waktu 24 (CT P1) baru bisa mulai, padahal ia datang di waktu 1.

Apakah ada kemungkinan *starvation* di SJF?

Ya, ada kemungkinan **starvation (kelaparan)** di SJF.

- *Starvation* terjadi ketika **proses dengan *Burst Time* yang sangat panjang terus-menerus terhalang** (tidak pernah mendapat CPU) karena selalu ada proses baru dengan *Burst Time* yang lebih pendek (atau sisa waktu yang lebih pendek) yang tiba di sistem.
- Hal ini sangat mungkin terjadi di sistem dengan beban kerja yang tinggi (banyak proses pendek baru terus datang).
- Solusi untuk mencegah *starvation* di SJF adalah dengan teknik **Aging**, yaitu meningkatkan prioritas suatu proses seiring bertambahnya waktu tunggu.

4. Real World Application

Untuk *web server* yang melayani banyak *request* kecil, algoritma mana yang cocok?

Round Robin (RR).

- *Web server* harus cepat merespons ribuan *request* (proses kecil) secara simultan.

- RR memastikan **Response Time yang cepat** dan **keadilan** (semua *request* dilayani secara bergiliran), memberikan pengalaman pengguna yang baik.

Untuk *render video* (task besar), algoritma mana yang cocok?

Shortest Job First (SJF) atau Prioritas Non-Preemptive.

- Jika kita tahu durasi *rendering* (BT) dan ingin menyelesaikan *render* tercepat terlebih dahulu, **SJF** optimal untuk throughput.
- Namun, untuk tugas yang benar-benar besar seperti *video rendering*, **Prioritas Non-Preemptive** (di mana tugas *rendering* diberikan prioritas rendah namun setelah dimulai tidak dapat disela) atau **FCFS** bisa digunakan jika *rendering* tersebut adalah satu-satunya tugas yang penting. **SJF** yang paling logis jika tujuannya adalah meminimalkan waktu penyelesaian tugas pendek-menengah.

Untuk OS *desktop* (interactive), algoritma mana yang cocok?

Round Robin (RR) dengan Skema Prioritas Dinamis.

- OS *desktop* (Windows, macOS, Linux) adalah sistem *time-sharing* yang sangat interaktif.
- **RR** digunakan sebagai dasar untuk memberikan **Response Time yang cepat** (misalnya, saat klik mouse atau mengetik).
- Algoritma modern menggabungkan RR dengan **Skema Prioritas** (prioritas tinggi untuk tugas *foreground/interactive* dan prioritas rendah untuk tugas *background*) dan **Prioritas Dinamis** (prioritas berubah berdasarkan perilaku proses) untuk memberikan *fairness* dan responsivitas terbaik.